

AI Healthcare Assistant — Hugging Face & Prompt Engineering

Short Report — MediAssist AI Pvt. Ltd. Assignment

1. Model Selection

The chatbot was built on Qwen/Qwen2.5-1.5B-Instruct, an instruction-tuned causal language model from the Qwen2.5 family, loaded through Hugging Face's `AutoTokenizer` and `AutoModelForCausalLM` classes and run via the Transformers text-generation pipeline.

This model was chosen for three practical reasons:

- Resource fit: at roughly 1.5 billion parameters (about 3 GB in fp16), it loads and runs comfortably on a free-tier Google Colab GPU, which was the target deployment environment for this assignment.
- Instruction-tuning: unlike a base language model, Qwen2.5-1.5B-Instruct is trained to follow chat-style instructions and supports a proper chat template through the tokenizer, which is essential for role prompting, system prompts, and reliable formatting.
- Open licensing: it is released under the Apache-2.0 license, which is straightforward for an internal prototype like this.

For comparison, Microsoft's Phi-3-mini-4k-instruct (about 3.8 billion parameters) was evaluated as an alternative in Task 6. It offers stronger reasoning and more reliable structured-output generation at the cost of higher VRAM and slower inference, making it a natural upgrade path once compute is less constrained. The final design keeps the model swappable: a single `MODEL_NAME` variable controls which checkpoint is loaded, so the underlying model can be changed without altering any prompt logic.

2. Prompt Engineering Techniques Used

Each task in the assignment targeted a specific failure mode of the original chatbot (inconsistency, hallucination, unstructured output) using a distinct prompting technique.

2.1 Role, Context, Constraints, Formatting (Task 2)

The vague prompt "Tell me about diabetes" was rewritten by assigning the model a role (board-certified endocrinologist / patient-education specialist), giving context (audience and purpose), adding constraints (no invented statistics, admit uncertainty, stay general rather than diagnostic), and enforcing a fixed Markdown output structure with five required headings. This combination directly addresses inconsistency and unstructured output.

2.2 Few-Shot Prompting (Task 3)

Clinical notes were converted into structured Symptoms/Severity summaries using three worked examples of increasing severity (Mild, Moderate, Severe) before the real input. Three examples, rather than one, were used deliberately: a single example risks the model treating the pattern as coincidental, while three establish both the output format and the severity scale the model should apply.

2.3 Chain-of-Thought Prompting (Task 4)

For the fever/cough/SpO2 = 88% scenario, the prompt instructed the model to reason step-by-step internally about what a low oxygen saturation implies before answering, while explicitly forbidding it from revealing that reasoning in the output. Only a fixed three-line block (Urgency Level, Key Concern, Recommended Action) was allowed as output. This captures the accuracy benefit of chain-of-thought reasoning without exposing raw model reasoning in a clinical interface.

2.4 Structured JSON Output (Task 5)

A strict schema (disease, symptoms, risk_level, recommendation) was enforced through explicit formatting rules, a temperature of 0 to minimize syntactic drift, and a regex-based extraction step that pulls the JSON object out of the raw text before parsing. A one-shot repair retry was added: if the first response fails to parse as valid JSON, the model is told so and asked to correct it. This is a practical reliability technique on top of prompt design alone.

2.5 Prompt Optimization for Hallucination Reduction (Task 7)

The poor baseline output ("Maybe malaria. Or maybe dengue. Or maybe COVID. I am not sure.") was diagnosed as lacking structure, justification, and an actionable next step. The rewritten prompt combined role prompting, explicit constraints against unstructured hedging, a required ranked differential-diagnosis format with per-item justification, and a mandatory recommended-next-step line, while still permitting the model to ask for missing information instead of guessing.

2.6 Role-Based Template (Bonus)

A single prompt template with a {role} slot was used to change tone and technical depth for Doctor, Nurse, Medical Student, and Patient audiences, while keeping universal safety rules (no definitive diagnosis, no fabricated statistics, always recommend professional evaluation) constant across all roles. This demonstrates behavior branching without maintaining separate models or separate codebases.

3. Challenges Faced

- Generation-config deprecation warnings: passing individual generation arguments (max_new_tokens, temperature, etc.) directly to the pipeline alongside the model's default generation_config produced deprecation warnings in newer Transformers versions; this was resolved by constructing a single GenerationConfig object.
- Chat template dependency: instruction-tuned behavior only worked correctly once prompts were passed through tokenizer.apply_chat_template rather than as raw strings; skipping this step caused disorganized, less instruction-following output.
- JSON reliability with a small model: the 1.5B model occasionally added explanatory text around the JSON object despite explicit instructions not to. This was mitigated with regex extraction of the outermost {...} block and a one-shot repair-and-retry prompt rather than relying on prompt wording alone.
- Hidden reasoning leakage: for the chain-of-thought task, the model sometimes partially surfaced its reasoning before the final structured block despite being told not to; this is a known limitation of smaller instruction-tuned models and was noted as a case where a larger model (e.g. Phi-3-mini) would likely be more compliant.
- Balancing verbosity and conciseness: the baseline system prompt produced longer, more hedging answers than needed for a clinical assistant; later prompts had to explicitly bound response length and format to keep output usable in a real interface.
- Compute constraints: staying within free-tier Colab GPU memory limited the choice of model size, which shaped the model-selection decision in Section 1 and constrained how much reasoning depth could be expected from any single prompt.

4. Improvements Made After Prompt Refinement

The clearest before/after contrast in this assignment is between the unconstrained baseline behavior (Task 1's generic system prompt, and the poor differential-diagnosis output analyzed in Task 7) and the refined, technique-driven prompts built in Tasks 2 through 7. The table below summarizes the improvement.

Aspect	Before (Task 1 baseline / Task 7 poor output)	After (refined prompts)
Structure	Free-form paragraphs, no fixed sections	Fixed Markdown headings / JSON schema / labeled fields

Diagnosis handling	"Maybe malaria. Or maybe dengue. Or maybe COVID. I am not sure."	Ranked differential list, each item justified by a specific symptom, plus a next step
Reasoning visibility	None, or raw uncontrolled rambling	Internal step-by-step reasoning encouraged, but hidden from final output
Hallucination risk	High — no constraint against invented facts/stats	Reduced — explicit rule to avoid fabricated statistics/drug names, admit uncertainty

Overall, moving from a single generic system prompt to task-specific prompts using role assignment, explicit constraints, enforced output formatting, few-shot examples, hidden chain-of-thought, and schema-validated structured output measurably reduced the three problems identified in the original chatbot: inconsistent responses became consistently structured, hallucination-prone answers became evidence-scoped and appropriately hedged only where genuinely uncertain, and unstructured free text became Markdown, JSON, or fixed-field output suitable for downstream use by doctors, nurses, students, and patients alike.